TP : Tableaux, piles et files

Objectif
À la fin de cette activité, vous serez capables de :

- utiliser des boucles ;
- manipuler des tableaux ;
- construire une pile ;
- construire une file.

1 Découverte de `range`

Exercice 1

```
for i in range(5):
    print(i)
```

```
for i in range(2, 7):
    print(i)
```

```
for i in range(10, 0, -1):
    print(i)
```

Pour chacun des trois programmes :

1. Quelles sont les valeurs prises successivement par `i` ?
2. Combien de fois la boucle est-elle exécutée ?

2 Découverte des tableaux

Pour créer un tableau de taille `size` contenant partout la même valeur `x`, on écrit :

```
tab = [x] * size
```

Exemple :

```
tab = [-1] * 5
print(tab)
```

Affiche

```
[-1, -1, -1, -1, -1]
```

Exercice 2

Exécuter :

```
tab = [-1] * 10
for i in range(10):
    tab[i] = i
print(tab)
```

1. Que contient `tab` ?

Exercice 3

Compléter :

```
tab = [-1] * 10
for i in range(10):
    tab[i] = ..................
print(tab)
```

pour obtenir :

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

puis

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

3 Déplacer des éléments dans un tableau

Les exercices suivants seront utiles pour construire les piles et les files.

Exercice 4

Exécuter :

```
tab = ["A", "B", "C", None, None]

for i in range(2, -1, -1):
    tab[i + 1] = tab[i]

print(tab)
```

1. Quel est le résultat ?
2. Que fait cette boucle ?
3. Modifier le programme précédent afin d'obtenir :

   ```
   [X,A,B,C,None]
   ```

Exercice 5

Compléter :

```
tab = ["A", "B", "C", "D", None]
for i in range(1, 4):
    tab[........] = tab[........]
print(tab)
```

pour obtenir :

```
[B,C,D,D,None]
```

4 Construire une pile

Dans un tableau, il est possible de modifier la valeur de n'importe quelle case :

```
tab[0] = 5
tab[4] = 12
tab[7] = 3
```

Nous allons maintenant construire une structure de données particulière à partir d'un tableau : une **pile**. Dans une pile, on ne peut pas accéder directement à n'importe quelle case. Toutes les opérations se font au niveau du **sommet** de la pile :

— ajouter un élément au sommet (`empiler`) ;

— retirer l'élément situé au sommet (`depiler`).

Une pile fonctionne comme une pile d'assiettes : **Dernier arrivé = premier sorti**. Voici un exemple d'exécution :

```
empiler(A)

[A, None, None, None, None]

empiler(B)

[B, A, None, None, None]

empiler(C)

[C, B, A, None, None]

depiler()

[B, A, None, None, None]
```

Exercice 6

1. Compléter le fichier `pile.py`.
2.

5 Construire une file

Une file fonctionne comme une file d'attente.

Premier arrivé = premier sorti

```
Arrivées : A B C

Sorties  : A B C
```

Compléter le fichier `file.py`.

Tester ensuite :

```
enfiler(1)
enfiler(2)
enfiler(3)

print(F)
```

6 Applications

Application 1

Utiliser votre pile pour inverser :

```
[1, 2, 3, 4]
```

afin d'obtenir :

```
[4, 3, 2, 1]
```

Application 2

Utiliser votre file pour simuler une file d'attente :

— arrivée d'un client ;

— service d'un client ;

— affichage de la file.

Bilan

Compléter :

Pile	File
Dernier arrivé, ...	Premier arrivé, ...
Ajouter : ...	Ajouter : ...
Retirer : ...	Retirer : ...